

What I ran, what I captured, and what the checkpoints prove

This is not only an architecture summary. It follows the actual work from the WSL terminal to the documented evidence.

WALKTHROUGH RULE

For every part: show the local action, show the captured result, then explain the checkpoint boundary.

EVIDENCE PATH

The story moves from my local machine to the checkpoint

1. LOCAL WSL

I ran the commands on Aloof, Ubuntu 24.04 on WSL2.

The terminal screenshots show the real commands and outputs.

2. CAPTURE

I saved representative screenshots for Linux, Docker, CI/CD, and Kubernetes.

3. CHECKPOINT

CHECKPOINT.md records what is complete, what is only optional, and where the local work happened.

4. EXPLAIN

I separate:

DONE

EVIDENCED

FUTURE

The Repl contains the code, documents, and evidence files; local WSL was the execution environment for Parts 3 and 4.

PART 1 - LOCAL LINUX WALKTHROUGH

I started by checking the Linux machine

SYSTEM_HEALTH.SH OUTPUT

```
drainmon@l0aof:~$ nano devops-exam/scripts/system_health.sh
drainmon@l0aof:~$ chmod +x devops-exam/scripts/system_health.sh
bash devops-exam/scripts/system_health.sh

=====
SYSTEM HEALTH REPORT
=====

[1] DATE & TIME
Thu Aug 6 11:52:56 CST 2026

[2] UPTIME
11:52:56 up 14 min, 1 user, load average: 0.02, 0.23, 0.15

[3] MEMORY USAGE
      total      used      free   shared buff/cache   available
Mem:   1.9Gi    588Mi    968Mi    3.1Mi    1.4Gi    1.4Gi
Swap:   0B         0B         0B

[4] DISK SPACE
Filesystem      Size  Used Avail Use% Mounted on
none            987M    0 987M   0% /usr/lib/modules/5.15.167.4-microsoft-standard-WSL2
none            987M    4.0M 987M   1% /mnt/wsl
drivers          465G   390G   76G  84% /usr/lib/wsl/drivers
/dev/sdb         1007G   2.2G 954G   1% /
none            987M    88M 987M   1% /mnt/wslg
none            987M    0 987M   0% /usr/lib/wsl/lib
rootfs           983M   2.4M 981M   1% /init
none            987M   496M 986M   1% /run
none            987M    0 987M   0% /run/lock
none            987M    0 987M   0% /run/shm
tmpfs            4.0M    0 4.0M   0% /sys/fs/cgroup
none            987M   76K 987M   1% /mnt/wslg/versions.txt
none            987M   76K 987M   1% /mnt/wslg/doc
C:\              465G   390G   76G  84% /mnt/c
D:\              50M    16M   35M  32% /mnt/d
H:\             932G   845G   87G   91% /mnt/h
tmpfs            198M    16K 198M   1% /run/user/1000

[5] TOP 5 PROCESSES BY CPU
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root        53  0.6  0.7 66428 16092 ?        Ss   11:38   0:05 /usr/lib/systemd/systemd-journald
root        1  0.2  0.6 22198 13404 ?        Ss   11:38   0:02 /sbin/init
root     1156  0.1  3.0 2014724 61048 ?        Ssl  11:42   0:01 /usr/bin/containerd
root     1267  0.0  5.0 2130632 101780 ?        Ssl  11:42   0:00 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
message+  145  0.0  0.2 9772 5300 ?        Ss   11:38   0:00 dbus-daemon --system --address=systemd: --nofork --nopidfile --s
systemd-activation --syslog-only

[6] CPU INFO
4
0.02 0.23 0.15 1/191 2795

=====
END OF REPORT
=====
drainmon@l0aof:~$ |
```

LOCAL_WSL_CAPTURE

WHAT I RAN

bash devops-exam/scripts/system_health.sh

The script reported date/time, uptime, memory, disk space, top processes, and CPU information.

PART 1 - LOCAL LINUX WALKTHROUGH

Then I used scripts to produce repeatable evidence

BACKUP.SH + LOG_ANALYSIS.SH

```
draimon@Aloof:~$ nano ~/devops-exam/scripts/backup.sh
draimon@Aloof:~$ chmod +x ~/devops-exam/scripts/backup.sh
bash ~/devops-exam/scripts/backup.sh
Starting backup: 20260806_122657
tar: Removing leading '/' from member names
/tmp/devops-exam/app/
/tmp/devops-exam/app/current.log
Backup saved: /tmp/devops-exam/backup/backup_20260806_122657.tar.gz
Old backups cleaned.
total 28K
-rwxr-xr-x 1 draimon docker 292 Aug  6 12:24 app.log.gz
-rw-r--r-- 1 draimon docker 16K Aug  6 12:23 archive.tar
-rw-r--r-- 1 draimon docker 437 Aug  6 12:23 archive.tar.gz
-rw-r--r-- 1 draimon docker 398 Aug  6 12:24 archive.zip
-rw-r--r-- 1 draimon docker 179 Aug  6 12:26 backup_20260806_122657.tar.gz
draimon@Aloof:~$ nano ~/devops-exam/scripts/log_analysis.sh
draimon@Aloof:~$ chmod +x ~/devops-exam/scripts/log_analysis.sh
bash ~/devops-exam/scripts/log_analysis.sh
=== LOG ANALYSIS REPORT ===
File: /tmp/devops-exam/logs/app.log

Total lines:
18

ERROR count:
4

WARNING count:
2

All ERROR lines:
2026-08-06 08:01:05 ERROR Database connection failed
2026-08-06 08:03:15 ERROR Timeout on /api/users after 30s
2026-08-06 08:06:30 ERROR Authentication failed for user: bob
2026-08-06 08:08:40 ERROR Database connection failed

Last 5 entries:
2026-08-06 08:05:25 WARNING Disk usage at 78%
2026-08-06 08:06:30 ERROR Authentication failed for user: bob
2026-08-06 08:07:35 INFO Health check passed
2026-08-06 08:08:40 ERROR Database connection failed
2026-08-06 08:09:45 INFO Scheduled job completed

=== END ===
draimon@Aloof:~$ |
```

LOCAL_WSL_CAPTURE

RESULT

backup.sh created a timestamped tar.gz archive.

log_analysis.sh found 4 ERROR lines and 2 WARNING lines, then displayed the newest entries.

Checkpoint status: Part 1 complete; 11 screenshots present.

PART 2 - LOCAL DOCKER WALKTHROUGH

Before building, I cloned and checked the real applications

API-SRC AND UI-SRC FILE CHECK

```
draimon@Allof:~/devops-exam/part2-docker$ ~/devops-exam/documentation/Part2-Docker-Containerization-Documentation.md
-bash: /home/draimon/devops-exam/documentation/Part2-Docker-Containerization-Documentation.md: No such file or director
y
draimon@Allof:~/devops-exam/part2-docker$ printf "\nAPI Backend files:\n"
find api-src -maxdepth 1 -type f -print | sort

printf "\nUI Frontend files:\n"
find ui-src -maxdepth 2 -type f -print | sort

API Backend files:
api-src/.dockerignore
api-src/.gitignore
api-src/README.md
api-src/database.py
api-src/main.py
api-src/models.py
api-src/requirements.txt
api-src/schemas.py

UI Frontend files:
ui-src/.eslintrc.json
ui-src/.git/HEAD
ui-src/.git/config
ui-src/.git/description
ui-src/.git/index
ui-src/.git/packed-refs
ui-src/.gitignore
ui-src/README.md
ui-src/app/favicon.ico
ui-src/app/globals.css
ui-src/app/layout.js
ui-src/app/page.js
ui-src/app/page.module.css
ui-src/components/DataTab.js
ui-src/jsconfig.json
ui-src/next.config.js
ui-src/package-lock.json
ui-src/package.json
ui-src/public/next.svg
ui-src/public/vercel.svg
ui-src/yarn.lock
draimon@Allof:~/devops-exam/part2-docker$
```

LOCAL_WSL_CAPTURE

WHY THIS STEP MATTERED

The Docker build contexts had to contain the actual FastAPI and Next.js source.

The checkpoint calls out the early missing-source problem and confirms the final build used the cloned source.

PART 2 - LOCAL DOCKER WALKTHROUGH

The first run failed; I corrected the database and rebuilt

FIRST COMPOSE ERROR

```

devops_api | File "/usr/local/lib/python3.11/site-packages/uvicorn/main.py", line 440, in main
devops_api | run()
devops_api | File "/usr/local/lib/python3.11/site-packages/uvicorn/main.py", line 609, in run
devops_api | config.load_app()
devops_api | File "/usr/local/lib/python3.11/site-packages/uvicorn/config.py", line 428, in load_app
devops_api | return import_from_string(self.app)
devops_api | *****
devops_api | File "/usr/local/lib/python3.11/site-packages/uvicorn/importer.py", line 22, in import_from_string
devops_api | raise exc from None
devops_api | File "/usr/local/lib/python3.11/site-packages/uvicorn/importer.py", line 19, in import_from_string
devops_api | module = importlib.import_module(module_str)
devops_api | *****
devops_api | File "/usr/local/lib/python3.11/importlib/_init_.py", line 126, in import_module
devops_api | return _bootstrap._gcd_import(name[level:], package, level)
devops_api | *****
devops_api | File "<frozen importlib._bootstrap>", line 1204, in _gcd_import
devops_api | File "<frozen importlib._bootstrap>", line 1176, in _find_and_load
devops_api | File "<frozen importlib._bootstrap>", line 1147, in _find_and_load_unlocked
devops_api | File "<frozen importlib._bootstrap>", line 690, in _load_unlocked
devops_api | File "<frozen importlib._bootstrap_external>", line 940, in exec_module
devops_api | File "<frozen importlib._bootstrap>", line 241, in _call_with_frames_removed
devops_api | File "/app/main.py", line 3, in <module>
devops_api | import models
devops_api | File "/app/models.py", line 2, in <module>
devops_api | from database import Base
devops_api | File "/app/database.py", line 10, in <module>
devops_api | engine = create_engine(DB_CONNECTION_STRING)
devops_api | *****
devops_api | File "<string>", line 2, in create_engine
devops_api | File "/usr/local/lib/python3.11/site-packages/sqlalchemy/util/deprecations.py", line 281, in warned
devops_api | return fn(*args, **kwargs) # type: ignore[no-any-return]
devops_api | *****
devops_api | File "/usr/local/lib/python3.11/site-packages/sqlalchemy/engine/create.py", line 617, in create_engine
devops_api | dbapi = dbapi_meth(**dbapi_args)
devops_api |
devops_api | File "/usr/local/lib/python3.11/site-packages/sqlalchemy/dialects/mysql/pymysql.py", line 116, in import_dbapi
devops_api | return _import_("pymysql")
devops_api | *****
devops_api | ModuleNotFoundError: No module named 'pymysql'
devops_api | exited with code 1

```

LOCAL WSL CAPTURE - PROBLEM

problem: missing pymysql dependency

FINAL HEALTHY COMPOSE RUN

[illegible]

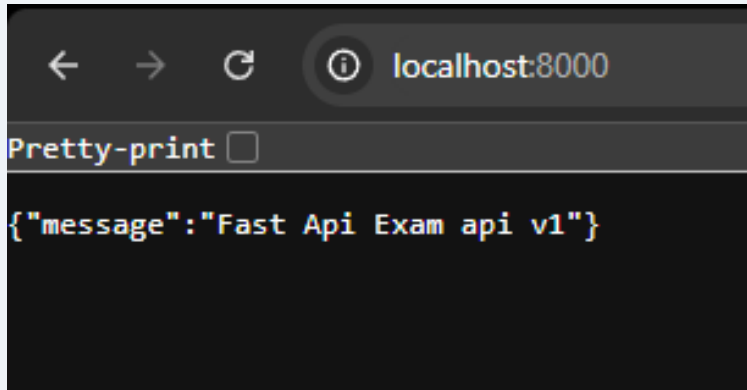
LOCAL WSL CAPTURE

```
result: API, MySQL, and UI Up (healthy)
```

PART 2 - LOCAL DOCKER WALKTHROUGH

Finally, I proved the stack responded

API ROOT RESPONSE



LOCAL WSL CAPTURE

LOCAL CHECKS

```
$ docker-compose ps

$ curl http://localhost:8000/
{"message": "Fast Api Exam api v1"}

$ curl -X POST /trip
{"name": "Docker Test Trip"}
```

CHECKPOINT RESULT

The final local walkthrough also recorded the /trip endpoint, a created test trip record, and the Next.js UI at localhost:3000.

PART 3 - LOCAL WSL TO GITHUB ACTIONS

I pushed staging and watched the remote pipeline run

LOCAL GIT PUSH

```
draiimon@Aloof:~/devops-exam$ git push origin staging
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 651 bytes | 651.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/draiimon/draiimon-devops-playground.git
 9642d8d..c84f2ee staging -> staging
draiimon@Aloof:~/devops-exam$ |
```

LOCAL WSL CAPTURE

GITHUB ACTIONS RUN HISTORY

Showing runs from all workflows

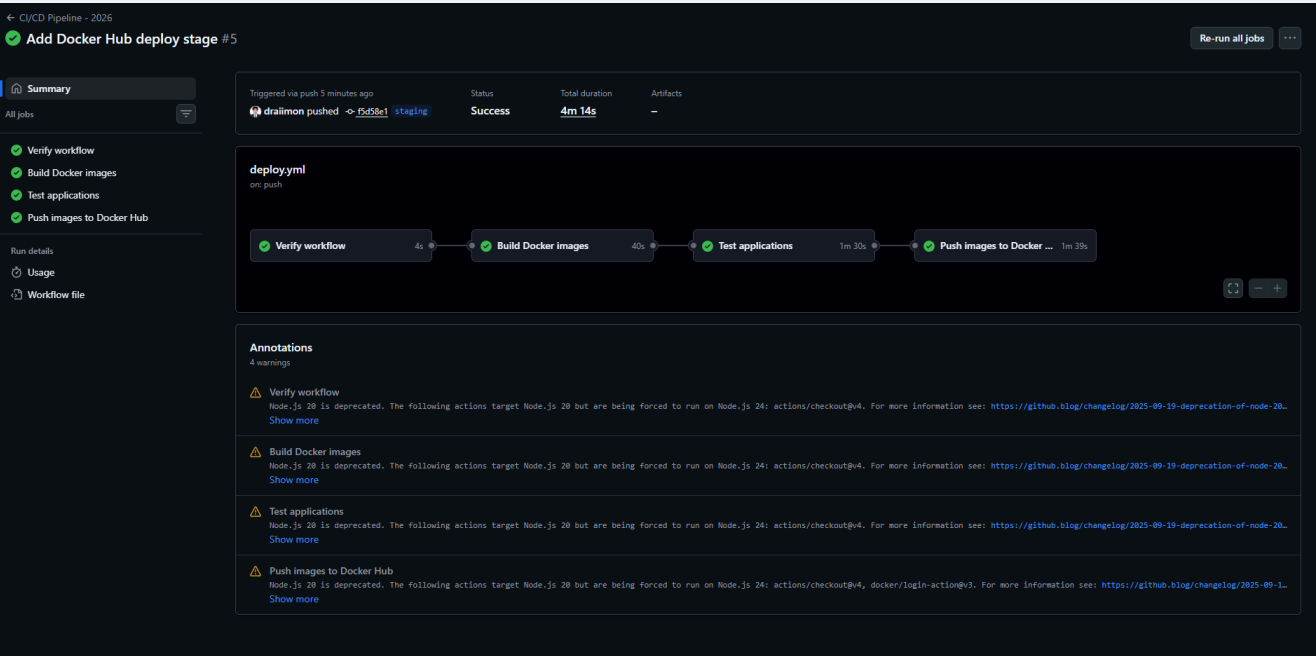
2 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Add Docker image build stage CI/CD Pipeline — 2026 #2: Commit 55c266 pushed by draiimon	staging		1 minute ago 1m 9s		...
✓	Add initial GitHub Actions workflow CI/CD Pipeline — 2026 #1: Commit c84f2ee pushed by draiimon	staging		Today at 9:17 AM 15s		...

CHECKPOINTED CI EVIDENCE

WALKTHROUGH: LOCAL PUSH -> GITHUB ACTIONS SUCCESSFUL RUNS

PART 3 - CI/CD WALKTHROUGH

SUCCESSFUL WORKFLOW GRAPH



CHECKPOINTED_GITHUB_EVIDENCE

WHAT THE CHECKPOINT CONFIRMS

Verify workflow
Build Docker images
Test applications
Push images to Docker Hub

A separate staging runtime used the published :staging images. Historical evidence shows a successful email; the current workflow also has a configured Discord path.

I prepared and deployed a two-node Minikube cluster

K9S: APPLICATION PODS READY

```
Context: minikube [RW]
Cluster: minikube
User: minikube
K9s Rev: v0.51.0
K8s Rev: v1.35.1
CPU: 0%
MEM: 0%

<0> all          <a> Attach      <ctrl-k> Kill      <o> Show ...
<1> devops-exam <ctrl-d> Delete    <l> Logs      <f> Show P
<2> default      <d> Describe <p> Logs Previous <t> Transf
<e> Edit         <shift-f> Port-Forward <w> Warp T
<?> Help        <z> Sanitize  <y> YAML
<shift-j> Jump Owner <s> Shell

pods(devops-exam)[5]
NAME↑ PF READY STATUS RESTARTS CPU %CPU/R %CPU/L MEM %MEM/R %MEM/L IP NODE AGE
api-app-77c77d88c9-bt9lm ● 1/1 Running 0 2 2 0 77 60 15 10.244.1.8 minikube-m02 6m24s
api-app-77c77d88c9-fb5tl ● 1/1 Running 0 2 2 0 58 45 11 10.244.1.6 minikube-m02 6m47s
db-7ff45cf46-7hmb2 ● 1/1 Running 0 6 6 1 444 173 57 10.244.1.3 minikube-m02 19m
ui-app-68f99bcd4-m4rp6 ● 1/1 Running 0 1 1 0 50 39 9 10.244.1.9 minikube-m02 5m44s
ui-app-68f99bcd4-vrrwf ● 1/1 Running 0 1 1 0 108 84 21 10.244.1.7 minikube-m02 6m47s
```

LOCAL WSL CAPTURE

WHAT I CHECKED

- Two API pods
- Two UI pods
- One MySQL pod
- All 1/1 Running
- Both Minikube nodes Ready

The checkpoint records Kubernetes v1.35.1, Minikube v1.38.1, Metrics Server, and Ingress enabled.

I tested traffic distribution and node failure

LOCAL WSL CAPTURE

LOCAL WSL CAPTURE

```
20/20 HTTP 200 while minikube-m02 NotReady
```

CHECKPOINT CLOSEOUT

What the evidence lets me claim

EVIDENCE-BACKED

Parts 1 and 2 are documented with captured core evidence.

Part 3 core pipeline and registry publishing are evidenced; the staging runtime is separately evidenced.

Part 4 runtime evidence is captured.

BOUNDARY

Parts 3 and 4 were executed on local WSL.

The Repl stores the code, documentation, screenshots, and checkpoint.

The local machine is not automatically changed by Repl edits.

FUTURE / OPTIONAL

Image security scanning.

Test the documented immutable-tag rollback.

Do not present these as completed evidence.

So the walkthrough is: run locally -> capture output -> compare with checkpoint -> explain exactly what is proven.

01. Opening

SAY

I will walk through the work the way I actually performed it: from the local terminal, to the captured screenshot, to the checkpoint that records the result. I will not treat a design diagram as proof by itself.

SHOW / DO

WALKTHROUGH ACTION

Start with the PDF, then say that the screenshots are the evidence trail.

EVIDENCE NOTE

The execution boundary matters: Parts 3 and 4 were run on my local WSL machine, while this repository contains the code and evidence.

02. Evidence path

SAY

Every part follows the same pattern. I ran something locally, saved the output, and then used the checkpoint to decide what can be claimed as complete. I also separate completed work from future improvements.

SHOW / DO

WALKTHROUGH ACTION

Point to the four boxes: local WSL, capture, checkpoint, explanation.

EVIDENCE NOTE

This prevents me from claiming that a planned scan or rollback test was already performed.

03. Part 1 system health

SAY

For Part 1, I began with the Linux environment on WSL2. I ran the system health script and captured the report with uptime, memory, disk, processes, and CPU information.

SHOW / DO

WALKTHROUGH ACTION

Zoom into the command at the top and the END OF REPORT marker.

EVIDENCE NOTE

The checkpoint identifies the machine as Aloof, Ubuntu 24.04 on WSL2.

04. Part 1 scripts

SAY

I then ran the backup and log-analysis scripts. The backup produced a timestamped archive, while the log script counted four errors and two warnings and printed the latest entries. This made the Linux work repeatable instead of only manual commands.

SHOW / DO

WALKTHROUGH ACTION

Show the backup filename and the ERROR/WARNING counts.

EVIDENCE NOTE

Part 1 is recorded as fully complete with four scripts and eleven screenshots.

05. Part 2 source

SAY

Before building containers, I checked that the actual API and UI source repositories were present. This was important because the first container attempt did not contain the real application source.

SHOW / DO

WALKTHROUGH ACTION

Point at `api-src/main.py`, `requirements.txt`, `ui-src/app`, and `package.json` in the screenshot.

EVIDENCE NOTE

The final Compose build contexts are `api-src` and `ui-src`, not empty wrapper folders.

06. Part 2 correction

SAY

The first Compose run failed because the API image did not have the MySQL Python driver. I corrected the dependency and aligned the database setup with the cloned application. The later screenshot shows all three services healthy.

SHOW / DO

WALKTHROUGH ACTION

Compare the `ModuleNotFoundError` screenshot with the final `docker-compose ps` output.

EVIDENCE NOTE

The legacy `docker-compose` command was used locally because that is what the WSL environment provided.

07. Part 2 response

SAY

After the containers were healthy, I tested the API root and the trip endpoint, and I checked the UI. The checkpoint records the final API, MySQL, and UI state and the successful test trip.

SHOW / DO

WALKTHROUGH ACTION

Show the browser response, then read the curl checks on the slide.

EVIDENCE NOTE

This is the difference between an image that builds and an application stack that actually responds.

08. Part 3 push

SAY

For Part 3, I pushed the staging branch from the local WSL checkout. That push is the handoff point: the local Git result is followed by the GitHub Actions run history.

SHOW / DO

WALKTHROUGH ACTION

Read the ``git push origin staging`` output and then point to the green staging runs.

EVIDENCE NOTE

The checkpoint says the workflow was created and executed on the candidate's personal WSL computer, with GitHub as the remote runner.

09. Part 3 workflow

SAY

The successful workflow graph shows the actual order: Verify, Build Docker images, Test applications, and Push images to Docker Hub. The checkpoint records a separate staging runtime and historical successful email evidence. The current workflow also contains a Discord notification job, but its delivery is not captured here.

SHOW / DO

WALKTHROUGH ACTION

Trace the four green jobs from left to right.

EVIDENCE NOTE

Node deprecation annotations are warnings, not failed jobs. Do not mix the historical email screenshot with the current Discord configuration. The optional image scan, failure-notification delivery, and live rollback remain follow-up work.

10. Part 4 deployment

SAY

For Part 4, I moved to Kubernetes locally. The k9s capture shows two API pods, two UI pods, and one database pod running, with the application replicas placed across the two Minikube nodes.

SHOW / DO

WALKTHROUGH ACTION

Point to READY 1/1, zero restarts, the pod names, and the node column.

EVIDENCE NOTE

The checkpoint records the two-node cluster, Kubernetes versions, enabled Metrics Server, and Ingress controller.

11. Part 4 runtime tests

SAY

I did not stop at a successful rollout. The verifier sent 40 requests and recorded 20 responses from each API replica. Then it stopped one Minikube node and still received 20 out of 20 HTTP 200 responses before restoring the node.

SHOW / DO

WALKTHROUGH ACTION

Read the successful requests and node-failure lines directly from the screenshots.

EVIDENCE NOTE

These are the strongest runtime claims because they come from captured local WSL evidence, not only YAML.

12. Closeout

SAY

The checkpoint lets me make precise claims. Parts 1 and 2 have captured core evidence. Part 3 core CI/CD and registry publishing are evidenced, with a separate staging runtime capture. Part 4 runtime evidence is captured. Security scanning, failure-notification delivery, and live rollback remain unverified.

SHOW / DO

WALKTHROUGH ACTION

Finish by opening CHECKPOINT.md and the documentation folders.

EVIDENCE NOTE

The final walkthrough is run locally, captured, reconciled against the checkpoint, and explained with the correct evidence boundary.